

# Community Health Worker (CHW) Assistance Platform

Comprehensive Project Blueprint: Complete Architecture, Tech Stack, Schemas, & Point-to-Point Operational Specifications

|                    |                                                                              |
|--------------------|------------------------------------------------------------------------------|
| Project Domain:    | Healthcare Informatics & Community Health Operations                         |
| Target Users:      | Community Health Workers (ASHA / ANM / PHC Auxiliary Nurses)                 |
| Architecture:      | 3-Tier RESTful Client-Server (React SPA + Express API + MongoDB)             |
| Authentication:    | JWT Bearer Tokens + Email OTP Verification + Bcrypt Password Hashing         |
| Core Modules:      | Patient Management, Appointment Scheduler, Health Records & Vitals, Security |
| Deployment Status: | Full Working Implementation (Verified End-to-End)                            |

## 1. Executive Summary & Clinical Problem Statement

Frontline community healthcare workers (such as ASHA workers and Auxiliary Nurse Midwives in rural and semi-urban communities) serve as the vital bridge between remote citizens and primary healthcare centers. Historically, health workers manage patient cohorts using physical paper logbooks. This approach presents critical bottlenecks: physical loss of maternal tracking records, missed infant immunization cycles, delayed vital triage for hypertensive and diabetic patients, and absence of unified clinical registries.

The **Community Health Worker Assistance Platform** bridges this digital gap by providing a reliable, web-first platform designed to streamline household registrations, schedule immunization and antenatal visits, monitor multi-parameter patient vitals, and maintain a permanent digital health record.

## 2. Complete Technology Stack & Tooling

The platform is engineered using modern, industry-standard web technologies characterized by high performance, asynchronous I/O, and maintainability:

| Technology Layer   | Component / Library     | Specific Role & Engineering Purpose                                                         |
|--------------------|-------------------------|---------------------------------------------------------------------------------------------|
| Frontend Core      | React.js 18 (SPA)       | Component-driven reactive user interface, state hooks, and virtual DOM rendering.           |
| Bundler & Runtime  | Vite 8.x                | Native ES module dev server, sub-second HMR, and tree-shaken production bundles.            |
| Client Routing     | React Router DOM v6     | Declarative client-side routing, protected route guards, and nested route history.          |
| API Client         | Axios with Interceptors | Promise-based HTTP client; automatically attaches JWT Bearer token to request headers.      |
| Styling System     | Modular Vanilla CSS3    | Custom healthcare color tokens, glassmorphism, responsive grids, and micro-animations.      |
| Server Runtime     | Node.js (ES Modules)    | Non-blocking, event-driven JavaScript engine running Express backend.                       |
| Web Framework      | Express.js 4.x          | RESTful endpoint routing, CORS configuration, JSON body parsing, and route handlers.        |
| Database & ODM     | MongoDB & Mongoose      | Schema-enforced NoSQL document persistence, subdocument nesting, and relational population. |
| Authentication     | JSON Web Token (JWT)    | Stateless cryptographic bearer token authentication for secure session verification.        |
| Password Security  | Bcrypt.js (10 Rounds)   | One-way cryptographic salt hashing of user passwords before saving to database.             |
| OTP Email Delivery | Nodemailer (SMTP)       | Automated delivery of 6-digit one-time passwords for worker account verification.           |

### 3. System Architecture & Database Schema Design

The system is partitioned into 5 normalized Mongoose data models linked via MongoDB ObjectIds:

#### 3.1 User Model (Health Workers & Administrators)

- **fullName**: String (trimmed, required) — Worker's official name.
  - **email**: String (lowercase, unique, indexed) — Primary login identifier.
  - **password**: String (minlength 6, bcrypt hashed via pre('save') middleware).
  - **phone**: String — Primary contact phone number.
  - **role**: Enum ['SUPER\_ADMIN', 'ADMIN', 'WORKER'] — Governs authorization gates.
  - **isEmailVerified**: Boolean — Set to true upon valid OTP submission.
  - **isApproved**: Boolean — Set to true for immediate account usage.

#### 3.2 Patient Model (Community Citizens)

- **fullName**, **age** (0-120), **gender** (Enum: ['MALE', 'FEMALE', 'OTHER']), **phone** (required).
  - **Demographics & Address**: address, village, district, state, pincode.
  - **Emergency Contact**: emergencyContactName, emergencyContactPhone.
  - **Medical Background**: bloodGroup (A+, A-, B+, B-, AB+, AB-, O+, O-), allergies, existingConditions.
  - **Audit Trail**: registeredBy (ObjectId ref: 'User'), status ('ACTIVE', 'INACTIVE').

#### 3.3 Appointment Model (Visits & Clinical Follow-ups)

- **patient**: ObjectId (Ref: 'Patient', required) — Associated patient.
- **worker**: ObjectId (Ref: 'User', required) — Assigned health worker.
- **appointmentDate** (Date), **timeSlot** (String, e.g., '10:00 AM').
- **purpose**: String (e.g., 'Antenatal Care Checkup', 'Child Immunization', 'BP Monitoring').
- **priority**: Enum ['LOW', 'MEDIUM', 'HIGH', 'URGENT'] — Clinical triage priority.
- **status**: Enum ['SCHEDULED', 'COMPLETED', 'CANCELLED', 'MISSED'].
- **location**: String (CHC, Village Anganwadi, Patient Home, Sub-Center), **notes**, **followUpDate**.

#### 3.4 Health Record Model (Vitals & Clinical Encounters)

- **patient**: ObjectId (Ref: 'Patient'), **recordedBy**: ObjectId (Ref: 'User').
  - **recordType**: Enum ['GENERAL\_CHECKUP', 'VITALS', 'ANTENATAL\_CARE', 'IMMUNIZATION', 'DIAGNOSIS', 'PRESCRIPTION', 'LAB\_TEST'].
  - **vitals Schema**: bloodPressure (mmHg), heartRate (bpm), temperature (°F), spO2 (%), bloodGlucose (mg/dL), weight (kg), height (cm).
  - **Clinical Assessment**: symptoms (String), diagnosis (String), treatmentPlan (String).
  - **medications Subdocuments**: Array of objects { name, dosage, frequency, duration }.

### 4. Point-to-Point Operational Workflows

Below is the exact execution flow of every primary operation implemented in the platform:

#### Operation 1: Health Worker Registration & Email OTP Verification

1. Health Worker fills registration form at /register (Full Name, Email, Phone, Password).
2. Frontend calls POST /api/auth/send-otp. Backend generates a 6-digit OTP stored in MongoDB with 10-min TTL.
3. Nodemailer sends verification email to user. Worker is redirected to /verify-otp.
4. Worker submits OTP -> POST /api/auth/verify-otp creates user account with bcrypt-hashed password and auto-approved status.

#### Operation 2: User Login & JWT Session Management

1. Worker submits Email & Password at /login.
2. POST /api/auth/login validates credentials via user.comparePassword(password) with bcrypt.

- 3. On success, server generates signed JWT token with userId and role.
- 4. Frontend stores token and user in localStorage. App routes are unblocked via ProtectedRoute.
- 5. Axios HTTP interceptor attaches 'Authorization: Bearer ' to all future outgoing requests.

### Operation 3: Dashboard Analytics & Metric Aggregation

- 1. Dashboard loads and executes concurrent Promise.allSettled requests to /patients, /appointments, and /health-records.
- 2. Calculates: Total Patients, Active status, Gender distribution (Male vs Female), Scheduled appointments, and Vital logs.
- 3. Interactive cards route directly to respective module pages with one click.

### Operation 4: Patient Registration, Real-time Search, and Editing

- 1. Worker clicks '+ Register Patient' -> Modal collects complete demographics, contact, emergency info, and medical history.
- 2. Submits POST /api/patients -> validated and saved to MongoDB with registeredBy audit link.
- 3. Instant multi-field search (/api/patients/search?q=) matches name, phone, village, or district using regex.
- 4. Worker can click 'Edit' in detail modal to update fields in-place via PUT /api/patients/:id.
- 5. Deep links: From patient details, worker can directly click 'Schedule Visit' or 'Add Health Record'.

### Operation 5: Appointment Scheduling, Prioritization & Lifecycle

- 1. Worker clicks '+ Schedule New Appointment' at /appointments.
- 2. Chooses patient, date, slot, priority (Urgent/High/Medium/Low), location, and purpose (quick tags: ANC, Immunization, BP & Sugar).
- 3. Submits POST /api/appointments -> saved with populated patient details.
- 4. Live filter tabs enable viewing All, Scheduled, Completed, or Cancelled appointments.
- 5. Single-click '✓ Complete' instantly triggers PUT /api/appointments/:id updating status in DB.

### Operation 6: Clinical Health Record & Multi-Vital Documentation

- 1. Worker opens /health-records and clicks '+ Create New Health Record'.
- 2. Selects patient and logs vitals: Blood Pressure, Pulse, Temp, SpO2, Blood Sugar, Weight.
- 3. Enters Clinical Symptoms, Diagnosis, and Treatment Plan.
- 4. Dynamic Prescription Builder: Worker clicks '+ Add Medicine' to prescribe drugs with dosage, frequency, and duration.
- 5. Submits POST /api/health-records -> stored and displayed with color-coded vital pills and full detail modal.

### Operation 7: Account Security & Change Password

- 1. From any page header, worker clicks '■ Password' button.
- 2. Modal opens with Current Password, New Password, and Confirm Password (with eye visibility toggles).
- 3. Frontend validates minimum 6 characters and password matching.
- 4. Submits PUT /api/users/change-password -> Backend validates current password hash with bcrypt and securely re-hashes new password.

## 5. Complete REST API Specifications

The platform exposes a comprehensive, RESTful API surface secured by JWT middleware:

| Method | Endpoint URL               | Access Gate | Description & Payload                               |
|--------|----------------------------|-------------|-----------------------------------------------------|
| POST   | /api/auth/send-otp         | Public      | Dispatches 6-digit OTP to provided email.           |
| POST   | /api/auth/verify-otp       | Public      | Verifies OTP code and registers verified user.      |
| POST   | /api/auth/login            | Public      | Authenticates credentials; issues signed JWT.       |
| GET    | /api/users/me              | Protected   | Returns current authenticated user profile.         |
| PUT    | /api/users/change-password | Protected   | Validates old password and hashes new password.     |
| GET    | /api/patients              | Protected   | Fetches all registered community patients.          |
| POST   | /api/patients              | Protected   | Registers a new patient record in MongoDB.          |
| GET    | /api/patients/search?q=    | Protected   | Searches by Name, Phone, Village, District.         |
| GET    | /api/patients/:id          | Protected   | Retrieves single patient demographic record.        |
| PUT    | /api/patients/:id          | Protected   | Updates existing patient information.               |
| GET    | /api/appointments          | Protected   | Lists appointments; supports status & date filters. |
| POST   | /api/appointments          | Protected   | Schedules new patient appointment or visit.         |
| PUT    | /api/appointments/:id      | Protected   | Updates appointment status (e.g. COMPLETED).        |
| DELETE | /api/appointments/:id      | Protected   | Cancels or removes scheduled appointment.           |
| GET    | /api/health-records        | Protected   | Lists clinical records; filters by patient or type. |
| POST   | /api/health-records        | Protected   | Saves patient vitals, diagnosis, and medications.   |
| DELETE | /api/health-records/:id    | Protected   | Deletes a specific health record.                   |

## 6. Step-by-Step Live Demonstration Runbook

Follow this sequential checklist during evaluation and viva presentations:

1. Open <http://localhost:5173> -> Redirects automatically to /login.
2. Click 'Create an Account' -> Enter details, receive OTP email, verify OTP, and login.
3. Review Dashboard: Observe logged-in health worker credentials, active patient counts, and module cards.
4. Navigate to Patient Management: Register a test community patient; search via name and phone number; edit patient record.
5. From Patient Details modal: Click 'Schedule Visit' -> Schedules an Antenatal Care or Checkup appointment.
6. Navigate to Appointments page: Filter by status, mark visit as '✓ Complete'.
7. Navigate to Health Records: Log Blood Pressure (120/80), Heart Rate (72), and SpO2 (98%); prescribe Paracetamol; save record.
8. Test Account Security: Click '■ Password' in header; change password and verify instant update.
9. Click Logout -> Protected routes immediately lock down session and redirect to /login.